Model report: Diabetes progression (Option A)

Author: Nathan Kenney

Target and Type: For every patient in the dataset there is 10 health measurements (X) which are paired with a provided target number (y) representing measured disease progression score one year later in diabetes. This progression score represents a continuous number as it is a numerical measurement measuring actual disease progression one year later per patient where the higher the number is, the further the disease has progressed and is not represented in a categorical format such as 1 or 0. Due to the fact that it's continuous, the data is represented as a regression problem making a linear regression model the best choice to use.

In [12]:
```
# Option A — Diabetes progression
from sklearn.datasets import load_diabetes

data = load_diabetes(as_frame=True)

# Splits data into two pieces where X represents the features table and y is the progression score
X, y = data.data, data.target
```

preprocessing

Here we check if the table contains any missing values from the diabetes progression dataset and then checks its categorical columns. Because no data was missing, we didn't have to drop any values from the table. Scaling was not needed in this case for preprocessing because ordinary linear regression models do not benefit from scaling and will provide the same predictions anyway. All columns are float64 so no encoding was needed.

In [13]:
```
# Check for missing values
print(X.isna().sum().sum())

# Look for categoral columns
print(X.dtypes.unique())
```

```
0
[dtype('float64')]
```

Split, Fit, Predict, and Evaluate

Our model results in a R^2 score of 0.45. This means that our linear regression model predicts progression of the disease with 45% less error than guessing the average score for every patient. In order to get this we have to split the data by 80/20 with random state 42 where we treat 20% of the dataset as if it were unseen by the model. This split is reasonable as it maintains the majority of the data where it knows how to represent the data, while still providing it a decent chunk to compare 'unseen' data. the 45% is a reasonable bump because it cannot account for changes that occur throught the year as its purpose it to take day 1 data and predict progression a year later. I would not trust the model for any specific patient due to the fact that much of the variation is unexplained, but it is still useful for spotting general trends across patients in the data.

In [14]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Splits the data using random seed 42
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)

# Model choice, Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)

print("R^2 on test set:", model.score(X_test, y_test))
```

```
R^2 on test set: 0.4526027629719195
```

Error analysis

The model predicted 71.62 for a patient whose actual score was 48. This is an overprediction of about 23.6. A plausible reason for this occurrence would be a lifestyle factor that occurred. Such as a change in diet for example that slowed progress down. These were not accounted for in our model.

In [11]:
```
import numpy as np
# Print the predicted and actual values of the dataset from a random person.
print("Predicted:", model.predict(X_test)[40])
print("Actual:", np.asarray(y_test)[40])
```

```
Predicted: 71.62331856151575
Actual: 48.0
```